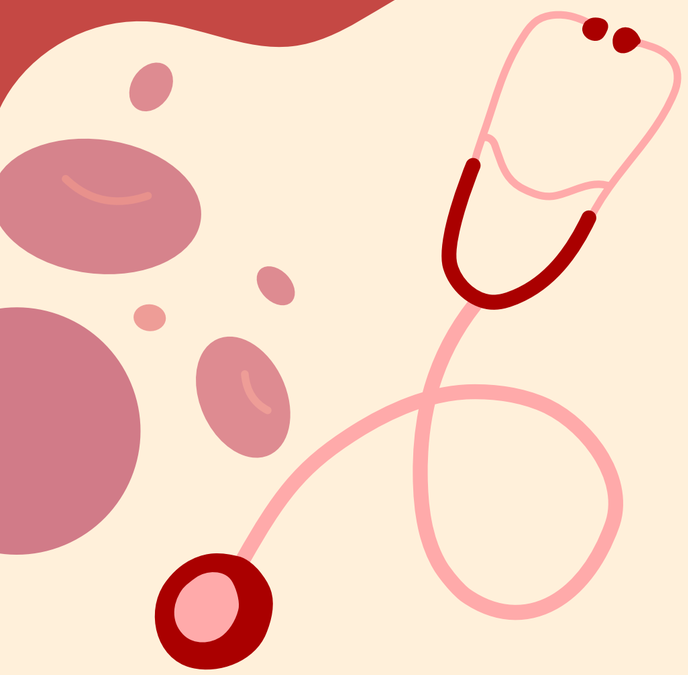
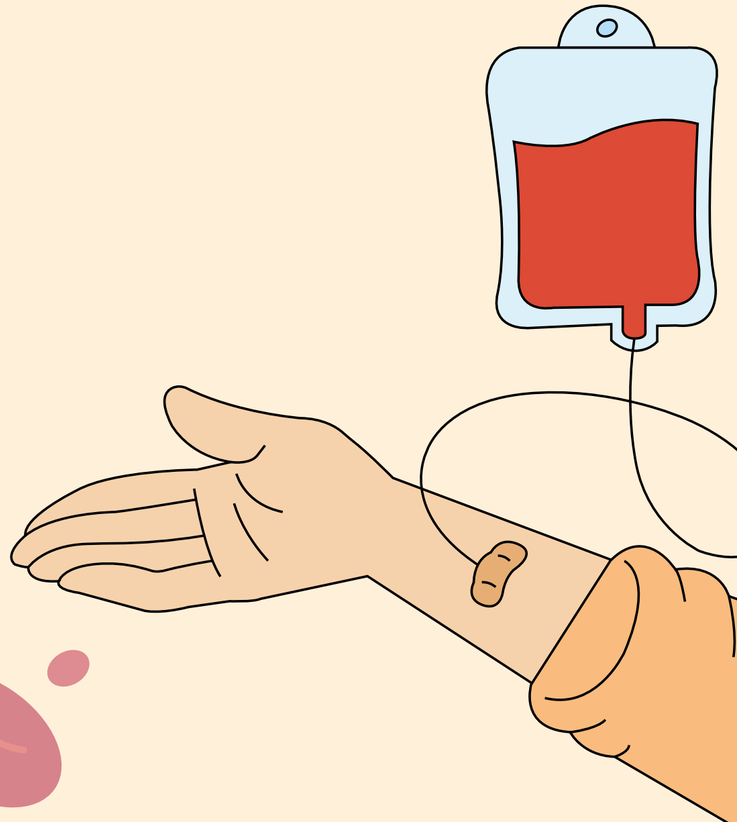
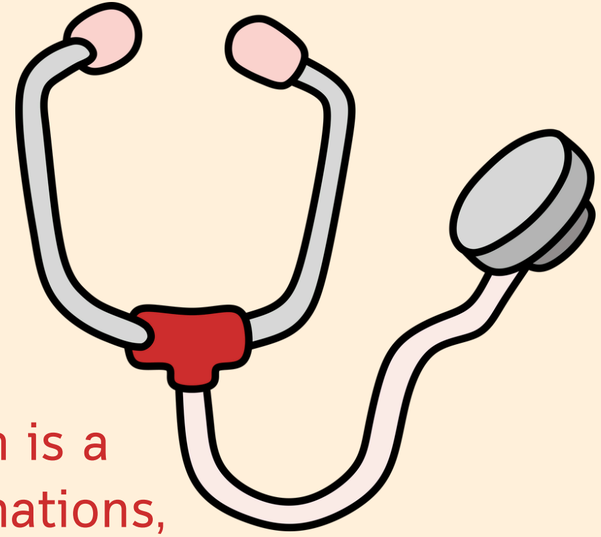
Java Project





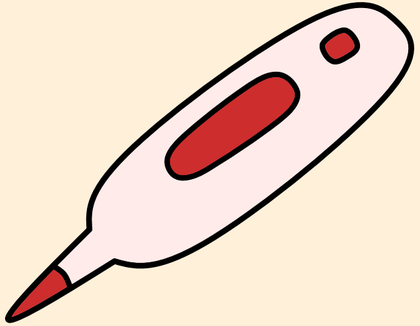


ABSTRACT

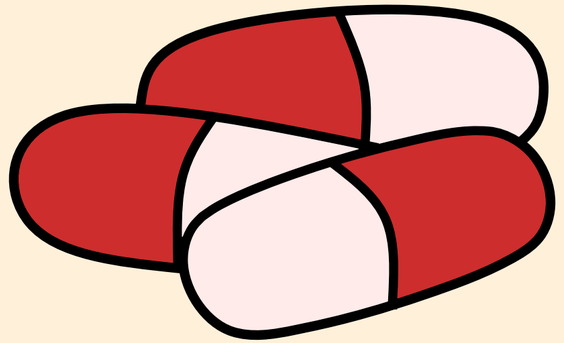


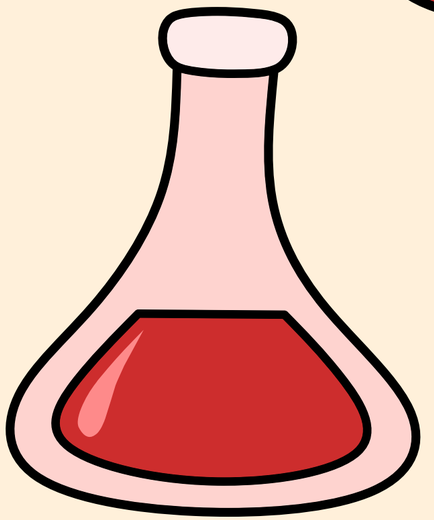
The Blood Bank Inventory and Repository System is a software application designed to manage blood donations, maintain blood stock records, and help identify suitable blood donors during emergencies. The system stores important information such as donor details, blood groups, available blood units, donation dates, and stock availability. It helps blood banks efficiently manage their inventory, reduce manual record-keeping, and retrieve information quickly when required. The system also supports searching for compatible blood groups and tracking blood stock levels to ensure better availability of blood for patients in need.





PROBLEM STATEMENT

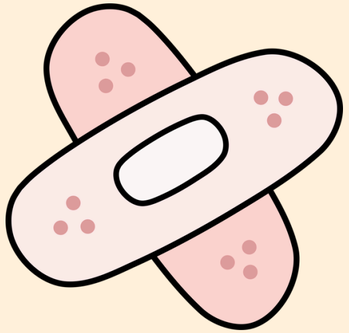




Traditional blood bank management systems often rely on manual record-keeping, making it difficult to maintain accurate blood stock details, manage donor information, and identify the availability of specific blood groups during emergencies. These methods can lead to delays in retrieving records, errors in inventory tracking, and difficulties in matching donors with patients who require blood.

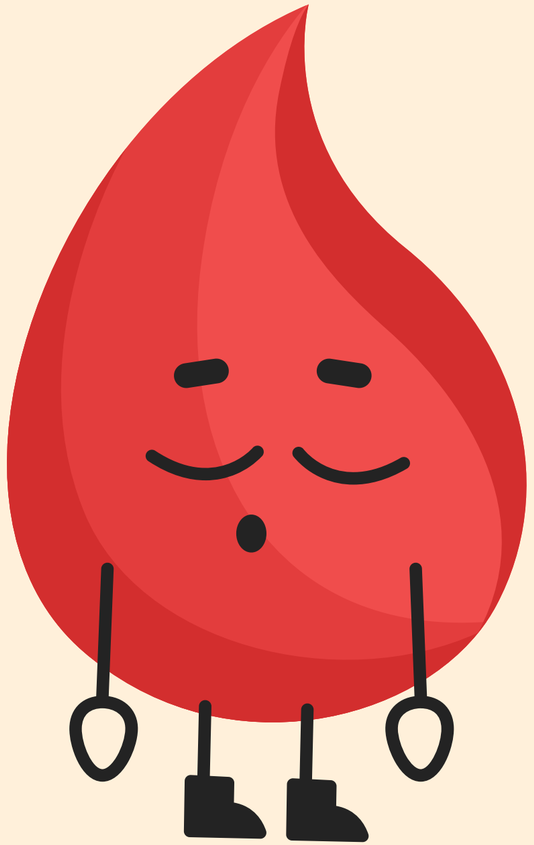
The Blood Bank Inventory and Repository System aims to develop a computerized solution using Java to store and manage donor details, blood group information, blood stock levels, and donation records. The system will enable users to add, search, update, and retrieve information efficiently, helping improve inventory management and support the timely availability of blood for patients in need.

C01 – Computational Thinking & Java Basics



1. IPO Model (Input-Process-Output)

- Input: Donor details, blood group, available stock, and blood requests.
- Process: Validate details, search for matching blood groups, and update inventory.
- Output: Updated stock report and blood request status.



2. Flowchart Idea

Start → Enter Request → Validate Request → Check Blood Stock → Is Stock Sufficient?

- Yes: Issue Blood → Update Inventory → End.
- No: Display Unavailable → End.

3. Algorithm / Pseudocode

1. Start.
2. Enter the required blood group and number of units.
3. Validate the request.
4. Check the available blood stock.
5. If sufficient stock is available, issue blood and reduce the stock.
6. Otherwise, display “Blood Not Available.”
7. Stop.

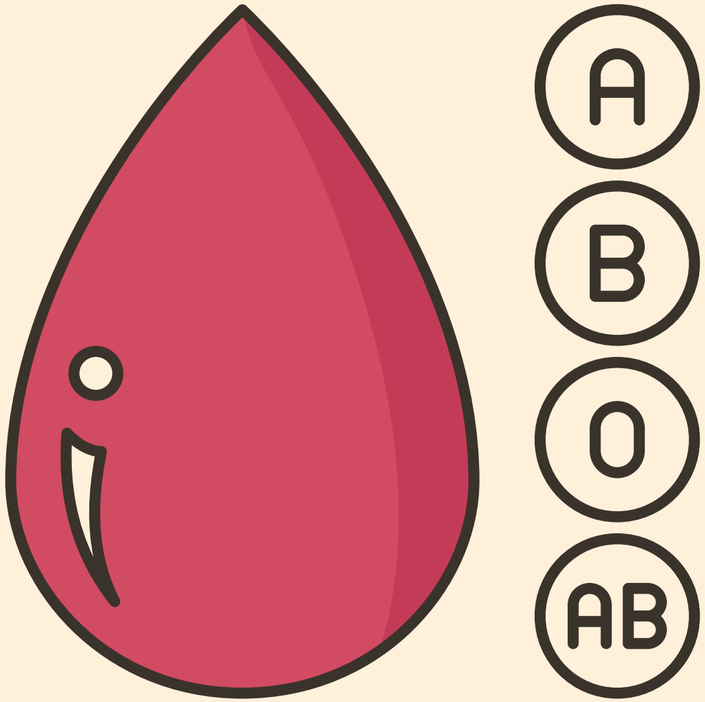
4. Java Program Structure

The project uses Java classes and the main() method to execute the program. Scanner is used to accept user input, while System.out.println() displays blood stock details and request results.

5. Variables and Operators

String: *Stores donor names and blood groups.*
int: *Stores the number of available blood units.*
LocalDate: *Stores donation dates.*
Arithmetic operators: *Update blood stock quantities.*
Relational and logical operators: *Validate requests and check stock availability.*

C02 – Arrays, Searching, Sorting & Matrices



1. One-Dimensional Arrays

Arrays are used to store multiple blood groups and their corresponding stock quantities.

Example: String[] groups = {"A+", "B+", "O+", "AB+"};

2. Array Traversal

Loops are used to access each array element, display blood stock details, and calculate the total number of available blood units.

3. Searching

Linear search is used to find a specific donor ID or blood group by checking each element until a match is found.

4. Sorting

Donor records can be sorted alphabetically by name, or blood stock entries can be arranged by blood group for easier viewing and management.

5. Two-Dimensional Array / Matrix

A two-dimensional array can represent blood stock according to blood group and blood component, such as whole blood, red blood cells, plasma, and platelets.

6. Practical Note

Arrays are suitable for storing and processing small amounts of data. For a larger real-world blood bank system, a database is more appropriate for secure, reliable, and efficient record management.

C03 – Classes, Objects & Encapsulation

1. Main Classes

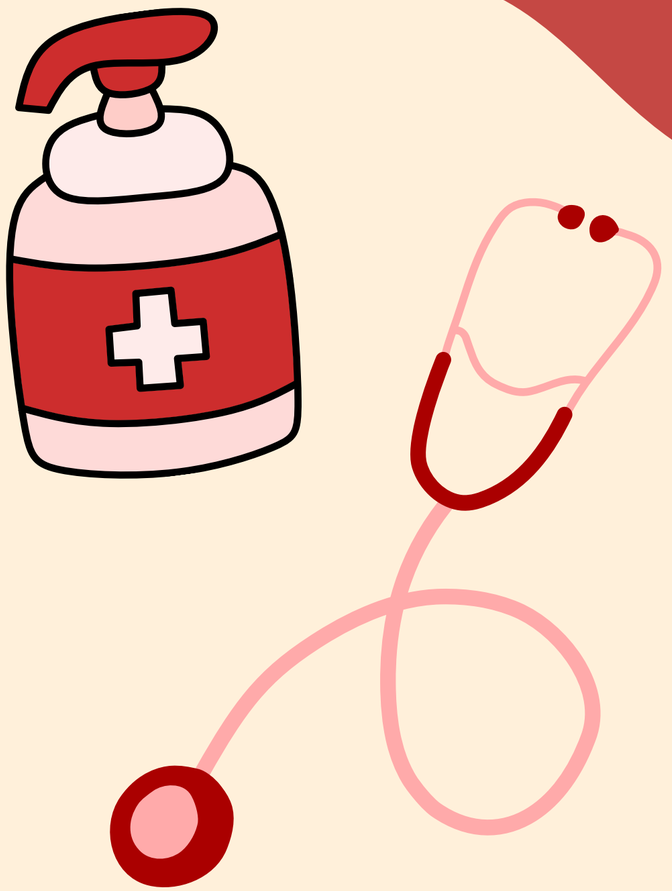
The system contains classes such as Donor, BloodUnit, BloodRequest, Inventory, and BloodBankService to manage donor details, blood stock, and requests.

2. Objects and Methods

Objects represent individual donors and blood units. Methods such as addDonor(), addUnit(), searchStock(), and issueUnits() perform specific operations.

3. Constructors

Constructors initialize donor details such as donor ID, name, blood group, and contact information when a donor object is created.



4. Encapsulation and Access Modifiers

Encapsulation protects sensitive data by keeping fields private and providing public methods to access or update information safely.

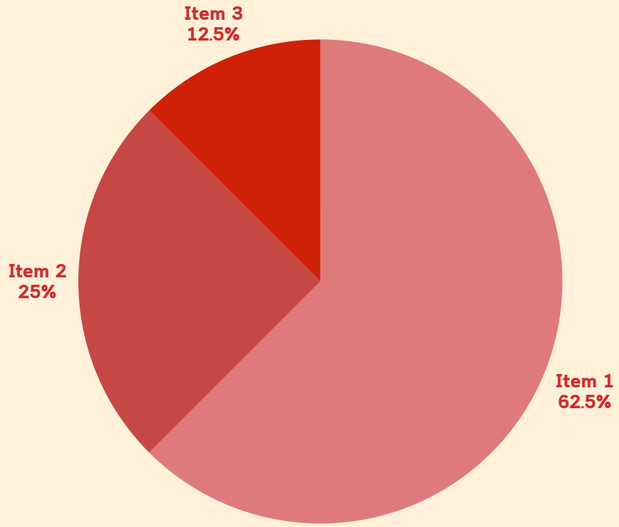
5. Static Members

A static counter can generate unique IDs for donor records without creating a separate counter for every object.

6. Object Relationships

The Inventory class manages multiple BloodUnit objects, while the BloodRequest class stores the required blood group and quantity.

C04 – Inheritance & Polymorphism



1. Inheritance

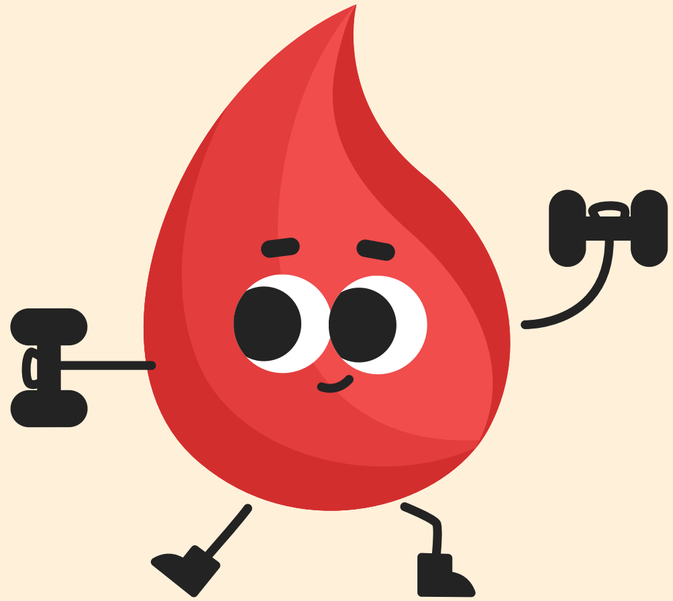
A common User class can be extended by StaffUser and Administrator classes to reuse shared login and profile details.

2. Polymorphism

Different user roles can implement the same viewDashboard() method with different behaviors according to their responsibilities.

3. Abstract Class

An abstract Record class can define common fields and an abstract validate() method for donor and inventory records.



4. Interface

A Searchable interface can declare a searchByGroup() method for searching donor records and blood inventory.

5. Packages

Packages such as model, service, repository, ui, and util organize the Java code into separate modules.

6. Runtime Binding

When an overridden method is called through a parent-class reference, Java selects the implementation according to the actual object at runtime.

C05 – Exceptions, Files & Collections

1. Exception Handling

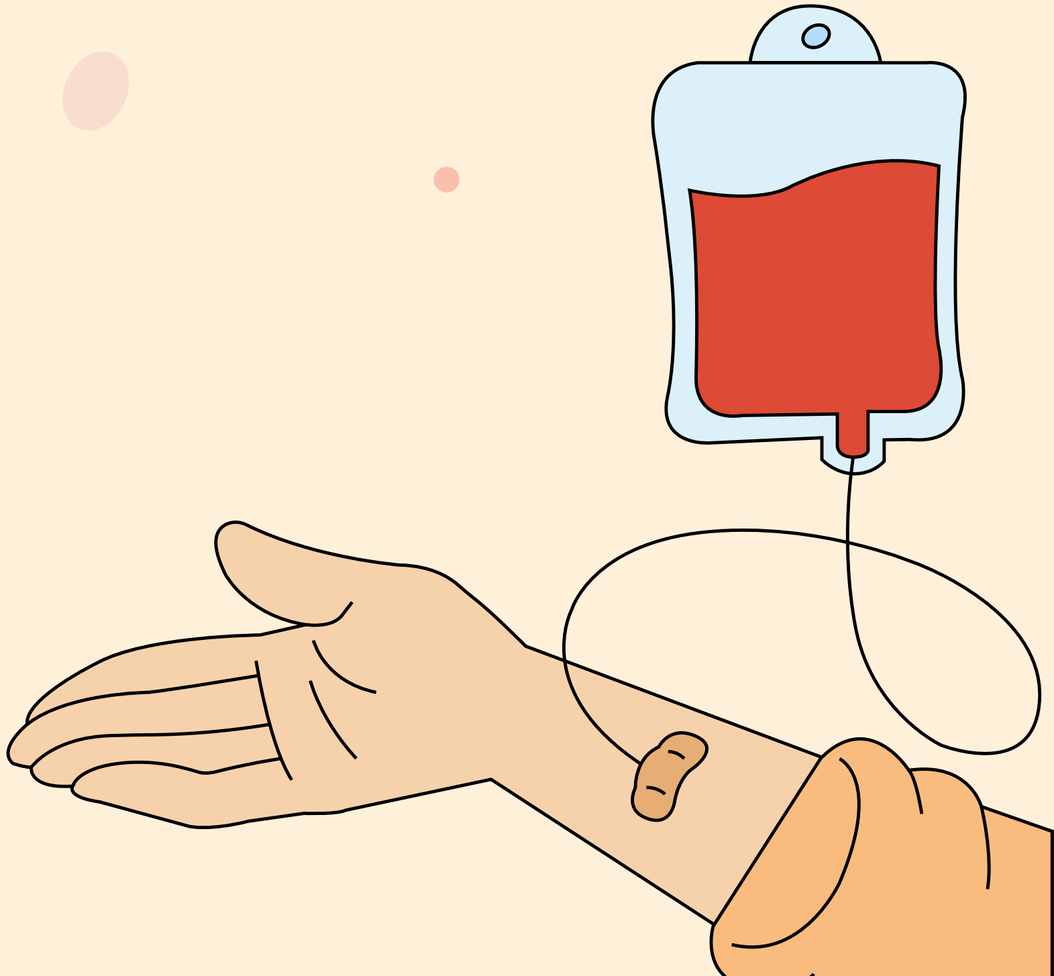
The system uses try-catch blocks to handle invalid input and file errors. Custom exceptions can indicate insufficient blood stock.

2. File I/O Streams

File handling allows the system to import and export donor details and inventory records using suitable Java file streams.

3. Collections and Generics

- ArrayList<Donor> stores multiple donor records.
- HashMap<String, Integer> stores blood groups and their available unit counts.
- Generics improve type safety when managing records.



4. Lambda Expressions

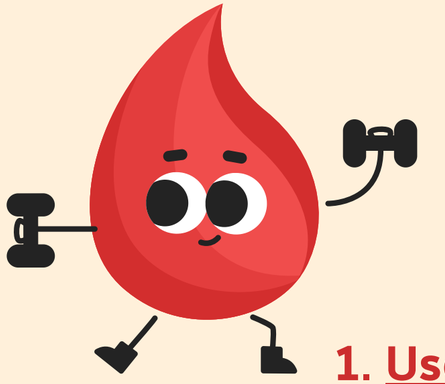
Lambda expressions can simplify sorting donor names alphabetically or arranging inventory records.

5. Stream API

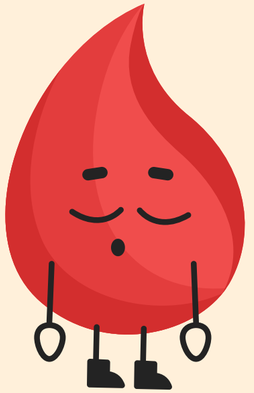
The Stream API can filter blood units by blood group and expiry status, then count or summarize the matching records.

6. Optional

Java's Optional class can represent a search result that may be absent, helping avoid unnecessary null checks.



C06 – SYSTEM ARCHITECTURE & WORKFLOW



1. User Interface

Staff members log in and select options such as Donors, Inventory, Blood Requests, and Reports to manage blood bank operations.

2. Service Layer

The service layer validates user inputs, checks permissions, applies business rules, and coordinates operations between the interface and data layer.

3. Repository / Data Layer

This layer stores and retrieves donor profiles, blood unit details, blood requests, and audit records using files or a database.

4. Main Workflow

Register Donation → Record Blood Units → Update Stock → Receive Blood Request → Check Availability → Authorize Issue → Update Stock and History.

5. Reports and Alerts

The system generates reports showing blood stock by group, low-stock warnings, near-expiry units, and pending or completed requests.

Architecture Flow:

User Interface

Staff login and dashboard

Service Layer

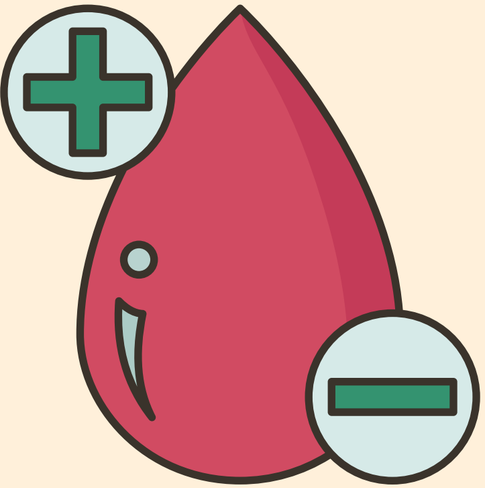
Validation and business logic

Repository / Database

Store and retrieve records

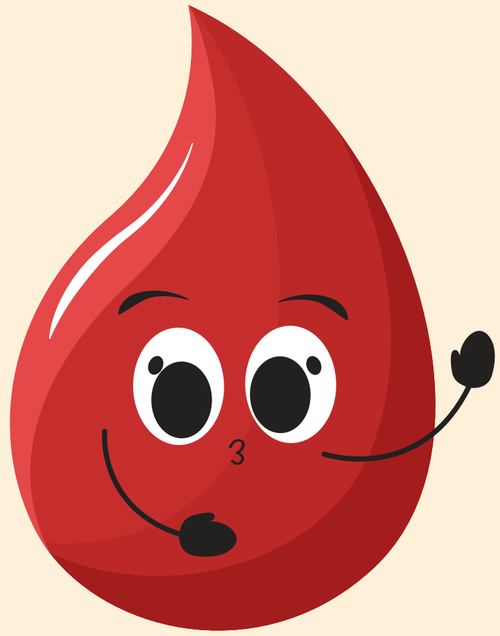
Results and Reports

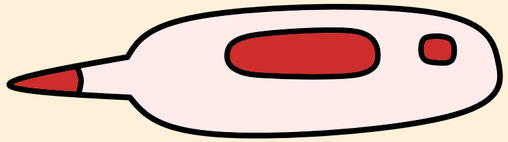
Stock updates, request status, and alerts



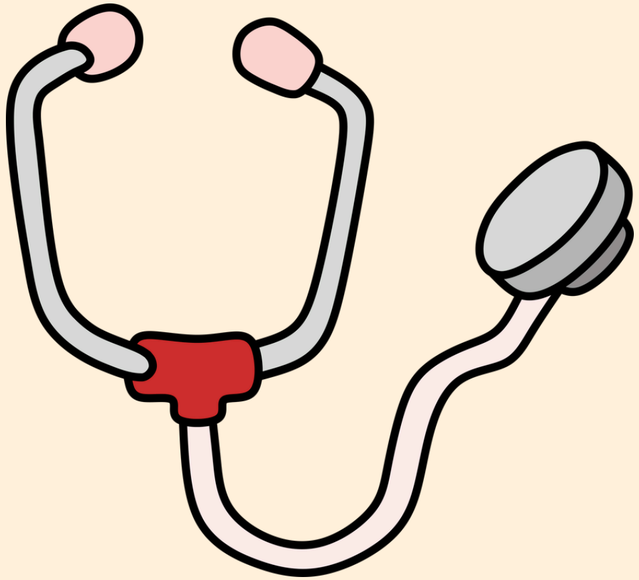
conclusion

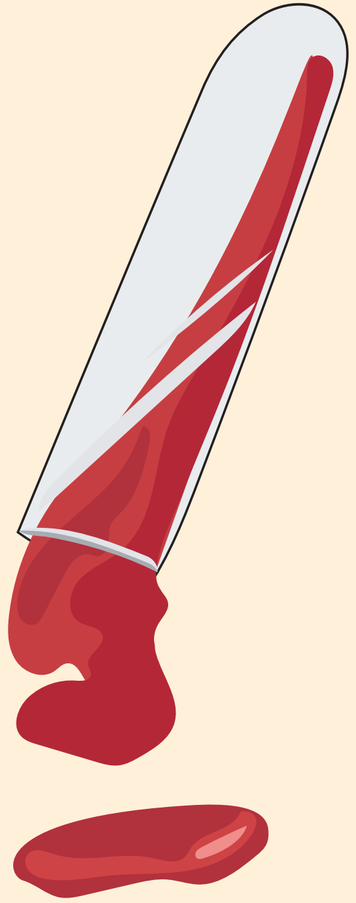
The Blood Bank Inventory and Repository System provides an efficient and organized way to manage blood donors, blood stock, and blood requests. By using Java programming concepts such as arrays, classes, objects, inheritance, exception handling, file handling, and collections, the system helps maintain records, track blood availability, and process requests systematically. It reduces manual effort, improves data accuracy, and supports faster access to essential information during emergencies. Overall, this project demonstrates how technology can improve blood bank management and contribute to the timely availability of blood for patients in need.

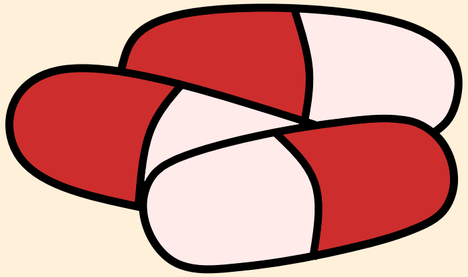




Thank You

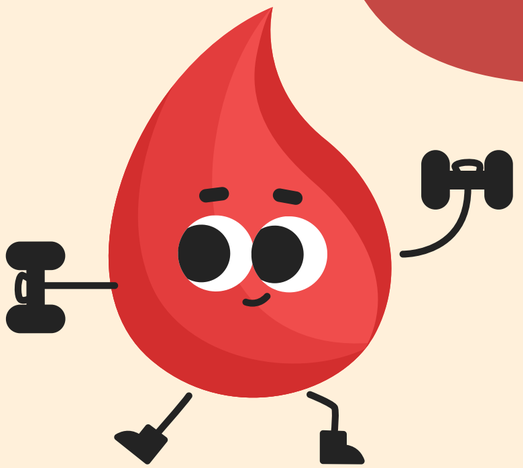






Project Partners





Yashwitha

Mahitha

